

Clarity — AI KPI Storytelling Engine

Accenture Innovation Challenge 2026 — Round 2 prototype

Problem statement: **BusinessIntelligence.ai** (Track 3)

Round 2 extends the Round 1 prototype into a KPI **intelligence-to-action engine**: multi-source/multi-cadence KPIs, a governed semantic contract, persona-specific narratives, an explicit sparse-history scenario, row/domain-level access control, a transparent LLM-vs-deterministic breakdown, real runtime telemetry (latency/tokens/cost), and a feedback-capture loop. See [§14 Round 2 requirements mapping](#) for the full checklist against the brief.

A dashboard can show revenue dropped 8%; it rarely explains why or what to do next. Clarity turns that gap into a button: click "Explain this KPI" and get a defensible, evidence-backed root-cause explanation with a confidence score and a recommended next step — in seconds, not days.

1. Project overview

Clarity is a working prototype of an **AI KPI storytelling engine**. It monitors a set of business metrics, automatically flags meaningful changes, attributes those changes to the dimensional segment(s) most responsible, pulls corroborating qualitative evidence, scores its own confidence, and writes an executive-ready natural-language explanation — with a fallback recommendation for what to do next.

It is built to demonstrate the full analytical loop the problem statement asks for:

```
INPUT (raw KPI time series)
→ AI PROCESSING (anomaly detection, root-cause attribution)
→ ANALYSIS (confidence scoring, evidence retrieval)
→ INSIGHT / EXPLANATION (natural-language narrative)
→ RECOMMENDED ACTION
→ BUSINESS IMPACT
```

2. Problem statement

See [docs/200-words.md](#) for the exact ~200-word problem statement and proposed solution used in the pitch deck.

3. Solution

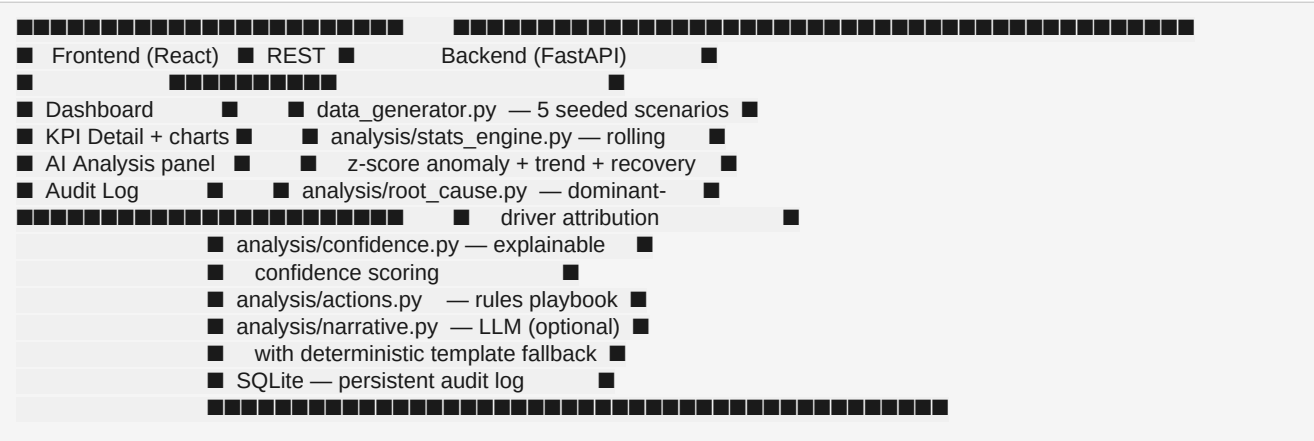
Clarity is a dashboard of monitored KPIs. Each KPI card shows its current value, trend, and a status (`normal` / `watch` / `critical` / `recovered`). Opening a KPI shows its 90-day trend with detected anomalies circled, and an **AI Analysis** panel with a single action: *"Explain this KPI."* Clicking it

runs the full pipeline and renders:

- A natural-language narrative of what changed and why
- A confidence score **with its reasoning spelled out**
- A ranked breakdown of contributing segments (bar chart)
- The qualitative evidence that corroborates (or fails to corroborate) the quantitative signal
- 2-3 concrete recommended next steps
- An explicit **ambiguity warning** when no single cause dominates — Clarity refuses to manufacture a confident-sounding answer when the data doesn't support one

Every analysis run is written to a persistent audit log (/audit page) — what was asked, what was concluded, how confident the system was, and whether the narrative came from an LLM or the offline template fallback.

4. Architecture



Hybrid AI, by design — not one model doing everything:

Task	Method	Why
Anomaly / trend detection	Rolling z-score + medium-term trend comparison (numpy)	Deterministic, auditable, no training data needed
Recovery detection	Rule-based V-shape heuristic	Cheap, explainable
Root-cause attribution	Rule-based contribution-share ranking	A dominant-driver decision should be reproducible, not probabilistic
Confidence scoring	Weighted formula over signal strength, data completeness, attribution clarity, evidence count	Every number ships with a plain-English reason
Recommended actions	Fixed playbook keyed by status/ambiguity	Action selection is exactly the kind of decision that should stay deterministic and auditable

Narrative generation	LLM (Anthropic Claude) if configured , else deterministic template	The LLM only phrases facts computed upstream — it never invents a number. If no API key is set (or the call fails/times out), Clarity falls back automatically so the demo never breaks
----------------------	---	---

5. Features implemented

- Live dashboard with 5 monitored KPIs, sparklines, and status badges
- 90-day interactive trend chart with anomaly markers
- One-click AI analysis with an animated processing pipeline
- Root-cause breakdown chart (contribution % by dimension)
- Evidence panel (simulated unstructured business context: CRM notes, support tickets, ops incident logs, product analytics)
- Confidence score with visible reasoning
- Explicit ambiguity handling (no forced diagnosis when data doesn't support one)
- Rule-based recommended-actions playbook
- Persistent, queryable audit log of every analysis run
- Responsive layout, loading/empty/error states
- Light/dark mode aware design tokens

6. AI methodology

See the architecture table above. In short: **rules and statistics decide what happened and what to do; an LLM (when available) is used only to phrase it for a human reader.** This directly reflects the challenge's "how do you move from correlation to something a business leader can act on, and what do you do when the data is genuinely ambiguous" prompts — the ambiguous-churn scenario in the demo data exists specifically to exercise that path.

7. Tech stack

- **Frontend:** React 19, TypeScript, Vite, Tailwind CSS v4, Recharts, React Router
- **Backend:** Python 3.11, FastAPI, SQLAlchemy, NumPy
- **Database:** SQLite (persistent audit log)
- **AI:** Anthropic Claude API (optional, narrative phrasing only) with a deterministic template fallback — the app is fully functional with zero external API keys

8. Setup

Backend

```
cd backend
python -m venv venv
./venv/Scripts/activate      # Windows; use `source venv/bin/activate` on macOS/Linux
pip install -r requirements.txt
cp .env.example .env         # optional: add ANTHROPIC_API_KEY to enable LLM narratives
uvicorn app.main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
cp .env.example .env         # defaults to http://localhost:8000, override if needed
npm run dev
```

Open <http://localhost:5173>.

9. Environment variables

backend/.env

Variable	Required	Default	Purpose
ANTHROPIC_API_KEY	No	(empty)	Enables LLM-phrased narratives. Omit to run fully offline in template mode.
ANTHROPIC_MODEL	No	claude-sonnet-4-5	Model used when the key is set
DATABASE_URL	No	sqlite:///./clarity_audit.db	Audit log storage
FRONTEND_ORIGIN	No	http://localhost:5173	CORS allow-list
WAREHOUSE_DATABASE_URL	No	(empty)	SQLAlchemy-compatible warehouse URL used by the live SQL connector
BI_API_URL	No	(empty)	BI tool REST endpoint used by the live connector probe
BI_API_TOKEN	No	(empty)	Optional bearer token for the BI tool connector

frontend/.env

Variable	Required	Default	Purpose
VITE_API_URL	No	http://localhost:8000	Backend base URL

No secret key is ever referenced from frontend code — `ANTHROPIC_API_KEY` is read only on the backend.

10. Running locally

1. Start the backend (`uvicorn app.main:app --port 8000`)
2. Start the frontend (`npm run dev`)
3. Open `http://localhost:5173` — the dashboard loads 5 live-generated KPIs
4. Click any KPI card → click **"Explain this KPI"**
5. Check **Audit Log** in the top nav to see the run logged

11. Demo workflow (6 scenarios, deliberately curated)

KPI	Status	What it demonstrates
Revenue — APAC	Critical	Sudden anomaly, one dominant root cause (enterprise account churn), high confidence
Orders — NA Online	Normal	Stable metric, nothing to explain — the system doesn't cry wolf
Conversion Rate — EMEA	Watch	Slow gradual decline with two overlapping factors (a checkout-latency regression + a broad-based channel decline) — the multi-factor scenario
Revenue — LATAM	Recovered	Dip caused by an incident, then a successful intervention — the system recognizes and documents the recovery
Churn Rate — Global SMB	Ambiguous	Weak, noisy signal with low data completeness — the system explicitly declines to name a dominant cause and lowers its confidence instead of guessing
Activation Rate — AI Copilot	Sparse-history	Feature launched 12 days ago — confidence is explicitly capped because there isn't enough history for a reliable baseline, not because the signal itself is weak

Two more scenarios live in the access layer rather than the data: ask for any KPI with a role that isn't entitled to it (e.g. `role=latam_manager` on `rev_apac`) to see the row/domain-level **security scenario** (403, not a silent empty result); switch personas on the same analysis to see the

role-based personalization scenario (executive vs. analyst vs. ops manager get different narratives from identical underlying facts).

12. Test instructions

Backend:

```
curl http://localhost:8000/api/health
curl http://localhost:8000/api/roles
curl "http://localhost:8000/api/kpis?role=global_exec"
curl "http://localhost:8000/api/kpis/conv_emea/contract" # semantic contract
curl -X POST "http://localhost:8000/api/kpis/rev_apac/analyze?persona=executive&role=global_exec"
curl -X POST "http://localhost:8000/api/kpis/rev_apac/analyze?persona=analyst&role=global_exec"
curl -X POST "http://localhost:8000/api/kpis/activation_ai_copilot/analyze?persona=analyst&role=analyst" # sparse-history
curl -i "http://localhost:8000/api/kpis/rev_apac?role=latam_manager" # expect 403 — RBAC scenario
curl -X POST "http://localhost:8000/api/feedback" -H "Content-Type: application/json" \
-d '{"kpi_id":"rev_apac","persona":"executive","useful":true}'
curl http://localhost:8000/api/audit-log
```

Frontend type-check / build:

```
cd frontend
npx tsc --noEmit
npm run build
```

Round 1 was verified end-to-end with a headless-browser pass. The Round 2 additions above were verified via: `tsc --noEmit + npm run build (clean)`, and live `curl` runs of every new endpoint (RBAC 403s, contract, persona narratives, sparse-history confidence capping, feedback capture) against a running `uvicorn` instance. A full browser click-through of the new UI (persona selector, role switcher, contract viewer, feedback buttons) has not yet been done — do that before final submission.

13. Limitations

- Demo data is synthetic (seeded, deterministic) — not connected to a real data warehouse or BI tool
- Anomaly detection is a rolling z-score + trend comparison; it is not seasonality-aware (a naive weekday-seasonality series was deliberately excluded from the demo data for exactly this reason — see comment in `backend/app/data_generator.py`)
- Interaction effects currently cover configured two-dimensional cuts; an exhaustive search over every possible high-cardinality dimension is not run
- No authentication — not needed for a single-tenant demo prototype
- LLM narrative mode calls a live API per request with no caching

14. Round 2 requirements mapping

Mapped against the BusinessIntelligence.ai Round 2 "Minimum Prototype Expectations." Built in a focused single push on top of the Round 1 engine — scoped honestly rather than padded; gaps are listed as gaps, not glossed over.

Requirement	Status	Where
3-5 connected KPIs across 2-3 sources with different grains/cadences	DONE	6 KPIs across 5 simulated source systems (Stripe+Salesforce, OMS stream, Amplitude+logs, PagerDuty) and 4 cadences (real-time streaming, hourly, daily, weekly batch) — <code>data_generator.py</code>
Lightweight KPI/semantic contract (definition, calculation, drivers, thresholds, lineage, access)	DONE	GET <code>/api/kpis/{id}/contract</code> — <code>routers/kpis.py</code> , <code>schemas.KPIContract</code>
≥2 personas with different narratives/actions	DONE	<code>executive / analyst / ops_manager</code> , prompt- and template-branched — <code>analysis/narrative.py</code>
One multi-factor KPI movement with known/simulated drivers	DONE	Conversion Rate — EMEA: checkout-latency regression <i>and</i> broad-based channel decline, tagged in <code>known_drivers</code>
One low-confidence / abstention scenario	DONE	Churn Rate — Global SMB: no dominant segment, confidence lowered, action defaults to "assign an analyst" rather than naming a cause
One sparse-history / newly-launched KPI scenario	DONE	Activation Rate — AI Copilot: 12 days of history, confidence explicitly capped in <code>analysis/confidence.py</code> ; also compared against a comparable-cohort benchmark rather than left as a bare low number — <code>KPIDetail.cohort_benchmark</code>
One role-based security/entitlement scenario	DONE	<code>access_roles</code> per KPI, enforced server-side (403, not client-side hiding) in <code>routers/kpis.py</code> + <code>routers/analysis.py</code> ; role switcher in the UI header
Evidence with source freshness, method, contribution, confidence, lineage	DONE	Evidence panel + confidence reasoning string (now including a staleness penalty derived from <code>refresh_cadence</code> — <code>analysis/confidence.py</code> <code>FRESHNESS_HOURS</code>) + contract's <code>lineage/refresh_cadence</code> fields
Clear LLM vs. non-LLM breakdown	DONE	<code>processing_steps</code> on every <code>AnalysisResult</code> , rendered in the UI — every stage tagged <code>deterministic</code> , <code>retrieval</code> , or <code>llm</code>

Runtime telemetry (latency, model calls, tokens, cost)	DONE	Real (not estimated) token counts read from the Anthropic API usage block when an LLM call is made; latency measured end-to-end; persisted per-run in the audit log
Mechanism to learn from feedback	DONE	POST /api/feedback persists useful/not-useful signals; each subsequent analysis consumes the KPI aggregate and trims confidence when at least 3 samples fall below a 50% useful-rate — FeedbackSignal makes the recalibration visible
Materiality = statistical significance and business impact	DONE	analysis/materiality.py blends significance (40%) with automated USD impact (60%) for every KPI using a governed per-unit value and documented basis in the KPI contract
Decision rights (can this role authorize the action, or must they escalate)	DONE	analysis/actions.check_decision_authority — checks the viewing role against the KPI's owner/access_roles; analyst is always advisory-only regardless of read access — AnalysisResult.decision_authority
KPI contract includes an accountable owner	DONE	owner field on every KPI, surfaced in /api/kpis/{id}/contract
Statistical "expected value" baseline (Actual vs. Expected)	DONE	analysis/stats_engine.expected_value — a linear trend-line fit to the pre-window history, projected forward; deliberately not Holt-Winters/seasonal so it stays auditable — AnalysisResult.expected_value / expected_deviation_pct
Row/column/domain-level security	DONE	Per-KPI entitlement remains server-enforced; security.py also applies field policy to calculation, lineage and raw evidence text, returning explicit redacted_fields metadata rather than relying on client-side hiding
Multi-dimensional root-cause search (interaction effects)	DONE	Conversion Rate — EMEA ranks traffic-source × device cells by contribution, movement and sample size and surfaces the leading interaction in the diagnosis and UI

Real data connectors (warehouse/BI tool)	PARTIAL credential-ready	<code>/api/connectors</code> provides real SQL and authenticated BI REST adapters plus connection probes. The checked-in demo uses seeded data because no enterprise URL/token is committed; set <code>WAREHOUSE_DATABASE_URL</code> , <code>BI_API_URL</code> , and <code>BI_API_TOKEN</code> to activate them
RAG over an evidence corpus	DONE	<code>analysis/evidence_retrieval.py</code> builds a TF-IDF vector index over <code>backend/evidence_corpus/documents.json</code> , performs cosine retrieval, and returns document ID, similarity score, freshness and lineage with every citation

15. Future roadmap

- Seasonality-aware anomaly detection (STL decomposition)
- Exhaustive three-plus-dimensional interaction search with cardinality controls
- OAuth-managed connector credentials and scheduled production ingestion
- Enterprise-scale embedding index with document-level ACL propagation
- Offline evaluation and model retraining beyond the implemented live confidence recalibration
- Slack/email alerting when a KPI crosses into `watch` or `critical`